



Filière : Ingénierie Logicielle (Cycle Licence)

Cours d'Analyse de Données 2 (Avec R)

Enseignant : Dr. Barry

Email : *kalidoubarry40@gmail.com*

Plan du cours : Analyse de données 2 (Avec R)

- **Chapitre 1** : Rappels et compléments méthodologiques
- **Chapitre 2** : Réduction de dimensionnalité
- **Chapitre 3** : Méthodes de sélection de variables
- **Chapitre 4** : Classification hiérarchique
- **Chapitre 5** : Classification non hiérarchique (K-means)
- **Chapitre 6**: Fondements et enjeux du Machine Learning

Chapitre 1 : Rappels et compléments méthodologiques

❖ Limites des modèles vus en AD1

Dans AD1, nous avons étudié principalement :

- 1) tests statistiques classiques
- 2) régression linéaire
- 3) régression logistique

Ces méthodes sont très utiles, mais elles ont des limites :

☞ Elles fonctionnent bien quand :

- 1) peu de variables
- 2) relations simples
- 3) données propres et structurées.

☞ Elles deviennent faibles quand :

- 1) beaucoup de variables explicatives
- 2) variables fortement corrélées
- 3) relations non linéaires
- 4) données volumineuses.



C'est pour cela qu'en AD2 nous allons voir :

- réduction de dimension
- sélection de variables
- méthodes de machine Learning.

❖ Problèmes clés :

1. Haute dimension
2. Sur-apprentissage (Overfitting)

- Problèmes en haute dimension

On parle de **haute dimension** quand :
nombre de variables $\approx$ nombre d'observations
ou nombre de variables $>$ nombre d'observations.



Conséquences :

- modèles instables
- multi colinéarité
- calculs difficiles
- mauvaise généralisation.



Solutions possibles

- ACP pour résumer l'information
- sélection des variables utiles

- Sur-apprentissage (overfitting)

Un modèle sur-apprend quand :

- il est très bon sur les données d'entraînement.
- mais mauvais sur de nouvelles données.

☞ Il a appris "par cœur".

Solution possible

Application de la validation croisée

Pour évaluer correctement un modèle, on ne doit pas utiliser les mêmes données pour :

- apprendre
- tester



k-fold cross validation:

- 1) On divise les données en k parties
- 2) On entraîne sur k-1 parties
- 3) On teste sur la partie restante
- 4) On répète k fois.

❖ Normalisation et standardisation

Les variables peuvent avoir des échelles différentes :

- âge : 18 → 60
- salaire : 3000 → 20000
- note : 0 → 20.

☞ Beaucoup d'algorithmes exigent des données à la même échelle.

Standardisation : $\frac{x - \text{moyenne}}{\text{Ecart-type}}$, En r `scale(data)`

Exercice d'application

- 1) Un enseignant a un jeu de données contenant :
25 étudiants
40 variables explicatives (notes, assiduité, âge, revenus des parents, etc.)
une variable à prédire : réussite (oui/non).
Pourquoi la régression logistique vue en AD1 n'est pas appropriée dans ce cas ?

- 2) On entraîne deux modèles pour prédire une note :

Modèle	Erreur entraînement	Erreur test
A	2 %	25%
B	15 %	18 %

Quel modèle est en sur-apprentissage ?

- 3) Un étudiant dit :
« J'ai utilisé toutes mes données pour construire le modèle, puis j'ai calculé l'erreur sur ces mêmes données. Mon modèle est excellent. »

Expliquez pourquoi cette démarche est incorrecte.
Proposez une procédure correcte basée sur : k-fold

Solution

- 1) ☞ Le nombre de variables est **supérieur** au nombre d'individus.
La régression logistique vue en AD1 n'est donc pas adaptée sans : la réduction de dimension
- 2) Le modèle A est en sur-apprentissage. En effet, Il est très bon sur les données d'entraînement (2 %), mais très mauvais sur de nouvelles données (25 %).
- 3) L'étudiant a :
utilisé **les mêmes données** pour : entraîner le modèle et tester le modèle.
Procédure correcte basée sur k-fold
Étapes :
 - a) Diviser les données en k parties (ex : k = 5).
 - b) Pour chaque itération :
 - entraîner le modèle sur k-1 parties
 - tester sur la partie restante.
 - c) Calculer :
 - l'erreur pour chaque itération
 - puis la moyenne des erreurs.

Chapitre 2 : Réduction de dimensionnalité

Dans de nombreux jeux de données réels, on dispose d'un **grand nombre de variables**. Cela pose plusieurs problèmes :

- difficulté d'interprétation
- redondance de l'information
- Multi colinéarité
- temps de calcul élevé
- risque de sur-apprentissage.

☞ **La réduction de dimensionnalité** consiste à **résumer l'information contenue dans plusieurs variables** à l'aide d'un nombre plus faible de nouvelles variables, tout en conservant l'essentiel de l'information.

2.1 Analyse en Composantes Principales (ACP)

- 2.1.1 Objectifs de l'ACP

L'ACP est une méthode de **réduction de dimension** appliquée aux **variables quantitatives**.

Ses objectifs sont :

- réduire le nombre de variables
- supprimer la redondance entre variables corrélées
- résumer l'information
- visualiser les données dans un plan de faible dimension (2D ou 3D).

☞ Les nouvelles variables obtenues sont appelées **composantes principales**.

Les étapes avant de réaliser une ACP

- Analyser la qualité des données:
 - ✓ Identification et traitement des valeurs manquantes
 - ✓ Imputation
- Normaliser les données
 - ✓ Mise a l'échelle des variables pour éviter l'effet taille(biais liés aux unités de mesure).
- Choix des variables grâce aux analyse bi variées
 - ✓ Elimination des variables redondante

Comment l'ACP transforme les données pour révéler l'informations essentielles ?

- ❖ Etape 1: Création d'une matrice de corrélation (ou covariance) a partir des données normalisée. Cette matrice capture les relation linéaires entre les variables.
- ❖ Etape 2: Décomposer la matrice de corrélation: trouver les valeurs propres et les vecteurs propres de la matrice.
 - Les vecteurs propres représentent la direction des composantes principales
 - Valeurs propres c'est la quantité de variance expliquée par chaque composantes.

Valeurs propres et inertie expliquée

Chaque composante principale est associée à :

- une **valeur propre**
- une **part d'inertie expliquée**.

Inertie expliquée

- Elle mesure la **quantité d'information conservée** par une composante.
- Plus la valeur propre est grande, plus la composante est importante.

☞ On conserve généralement les composantes qui expliquent **une grande part de l'inertie totale** (ex : 70–80 %).

Techniques pour choisir le nombre d'axes principaux

Le choix du nombre d'axes (ou composantes principales) est crucial pour interpréter correctement l'ACP.

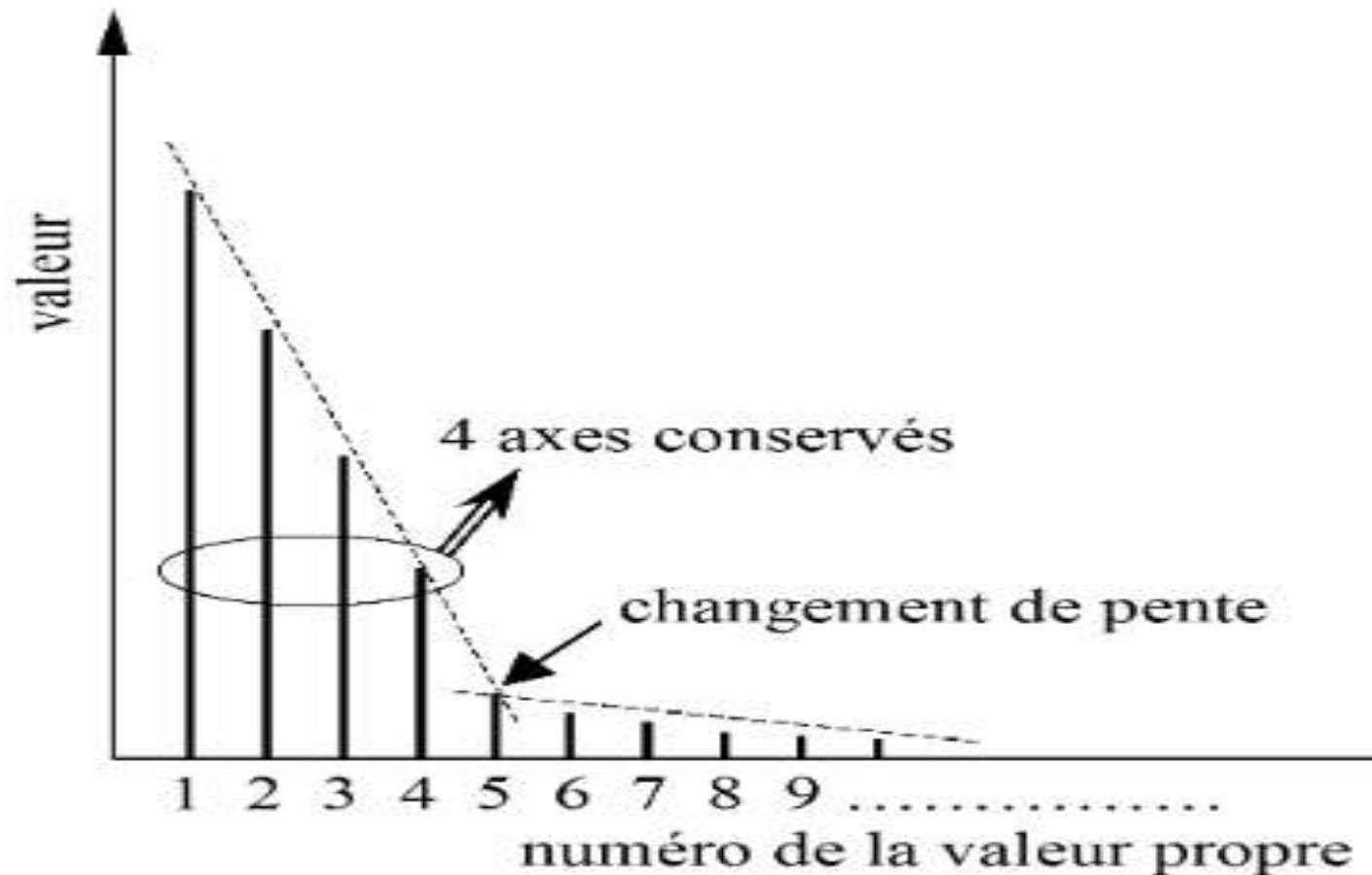
Voici les principales méthodes utilisées:

1. La règle de Kaiser.

-On conserve uniquement les composantes ayant une valeur propre > 1 .

2. Le graphe des éboulis (Scree Plot)

- On trace un graphique des valeurs propres et on cherche un point d'inflexion (méthode du coude)



Visualisation et interprétation

Graphique des Variables et des Individus



Projection des variables: On projette les variables initiales sur le plan des premières composantes principales en utilisant leurs coefficients



Cercle des corrélations : dessinez un cercle de rayon 1 autour de l'origine du graphique pour faciliter l'interprétation des relations entre les variables.



Corrélations entre variables : Plus une variable est proche du cercle des corrélations, plus est bien représentée par les composantes principales affichées.
Les variables proches les unes des autres sont corrélées positivement, tandis que celles diamétralement opposées sont corrélées négativement .



Contribution des variables aux axes : analysez la position des variables par rapport aux axes des composantes principales pour déterminer leur contribution à chaque axe. Les variables proches d'un axe contribuent fortement à la composante principale correspondante.

Contributions et $\cos^2$

a) Contribution

- La contribution mesure **l'importance d'une variable** dans la construction d'un axe.
- Une variable avec une forte contribution influence fortement l'axe.

b) $\cos^2$ (qualité de représentation)

- Le $\cos^2$ indique **à quel point une variable ou un individu est bien représenté** sur un axe.
- Plus le $\cos^2$ est proche de 1, meilleure est la représentation.

• Exercice d'application

1. Exécuter le code permettant de générer le tableau des notes.
2. Calculer et afficher la matrice de corrélation. Indice : **cor()**
3. Calculer et afficher la matrice de covariance. Indice : **var()**
4. Calculer les valeurs propres de la matrice de covariance. Indice : **eigen()**
et afficher :
Les valeurs propres → **eigen_decomp\$values**
Les vecteurs propres → **eigen_decomp\$vectors**
5. Effectuer une ACP avec centrage et réduction (**prcomp(notes, scale = TRUE)**)
6. Afficher le résumé de l'ACP.
7. Tracer le graphe des éboulis (**scree plot**).
8. Tracer la représentation simultanée des individus et des variables. Indice **biplot()**

2.2 Méthodes factorielles pour données qualitatives

- Lorsque les données sont **qualitatives**, l'ACP n'est pas adaptée. On utilise alors d'autres méthodes factorielles.

2.2.1 AFC – Analyse Factorielle des Correspondances

Principe

L'AFC est utilisée pour analyser un **tableau de contingence** (croisement de deux variables qualitatives).

Objectifs :

- étudier les relations entre modalités
- visualiser les associations
- réduire la dimension.

Interprétation

- Les modalités proches sur le graphique sont **associées**
- Les modalités éloignées sont **peu liées**.

• 2.2.2 ACM – Analyse des Correspondances Multiples

Principe

L'ACM est une généralisation de l'AFC à **plusieurs variables qualitatives**.

Elle est utilisée pour :

- Enquêtes
- Questionnaires

Interprétation graphique

- Les individus proches ont des profils similaires.
- Les modalités proches apparaissent souvent ensemble.
- Les axes résument les grandes oppositions entre profils.

le tableau récapitulatif des outils et fonctions R

Outil / Fonction	Rôle
Package FactoMineR	Bibliothèque principale pour effectuer les calculs de l'Analyse Factorielle.
Package factoextra	Bibliothèque utilisée pour extraire et visualiser les résultats de manière élégante.
Matrix	Permet de créer le tableau de contingence (le croisement de vos données).
Fonction CA()	Fonction qui exécute l'AFC
Fonction MCA()	Fonction qui exécute l'ACM
Fonction fviz_ca_biplot()	Crée la carte visuelle

Exercice d'application 1

1) Préférences de loisirs selon la filière d'étude

On a interrogé 120 étudiants sur leur filière et leur loisir principal.

Le tableau de contingence obtenu est le suivant:

Filière \ Loisir	Sport	Lecture	Jeux Vidéo	Musique
IA & Data	20	15	25	10
Génie Logiciel	10	30	5	15
Réseaux et Télécommunications	15	10	20	15

- a) Créer le tableau dans R
- b) Réaliser une AFC
- c) Afficher :
 - Les valeurs propres
 - Le pourcentage d'inertie expliquée
- d) Tracer le plan factoriel (dimensions 1 et 2)
- e) Interpréter les résultats

Exercice d'application 2

Etudiant	Filiere	Langage	Projet
1-10	IA	Python	Machine Learning
11-20	Reseaux	C++	Cybersecurite
21-30	Genie Logiciel	Java	Web

On considère 30 étudiants décrits par trois variables qualitatives :

- Filière
- Langage de programmation préféré
- Type de projet préféré

Les données sont résumées dans le tableau ci-dessus.

On demande :

- 1) D'effectuer une Analyse des Correspondances Multiples (ACM).
- 2) De représenter les modalités sur le plan factoriel.
- 3) D'interpréter les associations entre modalités.

Chapitre 3 : Méthodes de sélection de variables

Dans un jeu de données réel, il est fréquent d'avoir :

- un grand nombre de variables explicatives,
- des variables redondantes ou peu informatives,
- des variables fortement corrélées entre elles.

☞ Utiliser toutes les variables conduit souvent à :

- des modèles complexes,
- un risque de sur-apprentissage,
- une mauvaise interprétation des résultats.

La sélection de variables consiste à choisir **un sous-ensemble de variables pertinentes** afin de construire des modèles **simples, stables et interprétables**.

3.1 Approches classiques de sélection de variables

Ces méthodes sont basées sur des **modèles statistiques classiques** (régression linéaire ou logistique).

3.1.1 Sélection Forward (sélection progressive)

Principe:

- On commence avec un **modèle vide** (sans variable explicative).
- À chaque étape, on ajoute la variable qui améliore le plus le modèle.
- Le processus s'arrête lorsqu'aucune variable n'améliore significativement le modèle.

Avantages:

- Simple à comprendre.
- Utile quand le nombre de variables est élevé.

Inconvénients

- Une variable ajoutée ne peut plus être retirée.
- Peut conduire à un modèle sous-optimal.

Exercice d'application

On considère le jeu de données mtcars disponible dans R.

On s'intéresse à l'étude de la variable mpg (consommation de carburant) en fonction des variables explicatives suivantes :

- disp : cylindrée du moteur
- hp : puissance du moteur
- wt : poids du véhicule
- qsec : temps au quart de mile

On charge les données et on crée le dataframe suivant :

```
data(mtcars)
```

```
df <- mtcars[, c("mpg", "disp", "hp", "wt", "qsec")]
```

A) Sélection Forward automatique

1. Construire le modèle nul (sans variables explicatives).
2. Construire le modèle complet contenant toutes les variables explicatives.
3. Appliquer la méthode de sélection Forward automatique en utilisant la fonction **step()**.
4. Afficher le modèle final **summary(forward_model)**

B) Sélection Forward manuelle

- Comparer ce modèle avec celui obtenu par la méthode automatique.

3.1.2 Sélection Backward (élimination progressive)

Principe

- On commence avec le **modèle complet** (toutes les variables).
- À chaque étape, on supprime la variable la moins significative.
- Le processus s'arrête lorsque toutes les variables restantes sont pertinentes.

Avantages

- Considère l'effet de toutes les variables.
- Plus fiable que le forward si le nombre d'observations est suffisant.

Inconvénients

- Impossible si le nombre de variables est très grand par rapport aux observations.

3.1.3 Méthode Stepwise(Mixte)

Principe

- Combinaison des méthodes forward et backward.
- À chaque étape :
 - on peut ajouter une variable,
 - ou supprimer une variable déjà présente.

L'algorithme de l'approche:

- **Initialisation** : Partir d'un modèle vide
- **Phase d'Ajout (Forward)** : Tester l'ajout de chaque variable non présente.
- **Phase d'Élimination (Backward)** : Après chaque ajout, examiner toutes les variables déjà dans le modèle. Si l'une d'elles est devenue non significative suite à l'introduction de la nouvelle variable, elle est **supprimée**.
- **Test d'arrêt** : L'algorithme s'arrête lorsqu'aucune variable ne peut plus être ajoutée pour améliorer le modèle ET qu'aucune variable présente ne peut être retirée.
- **Itération** : Répéter le cycle ajout/retrait jusqu'à stabilisation complète du modèle.

3.1.4 Critères de choix des modèles

Pour comparer plusieurs modèles, on utilise des **critères quantitatifs**.

a) **AIC – Akaike Information Criterion**

Mesure le compromis entre :

- qualité d'ajustement,
- complexité du modèle.

Plus **l'AIC est faible**, meilleur est le modèle.

b) **BIC – Bayesian Information Criterion**

- Similaire à l'AIC mais pénalise **plus fortement** les modèles complexes.
- Plus **le BIC est faible**, meilleur est le modèle.

c) **R² ajusté**

- Variante du R² classique.
- Tient compte du nombre de variables.
- Peut **diminuer** si on ajoute une variable inutile.

Exercice d'application

On considère le jeu de données mtcars disponible dans R.

On s'intéresse à l'étude de la variable mpg (consommation de carburant) en fonction des variables explicatives suivantes :

- disp : cylindrée du moteur
- hp : puissance du moteur
- wt : poids du véhicule
- qsec : temps au quart de mile

On charge les données et on crée le dataframe suivant :

```
data(mtcars)
```

```
df <- mtcars[, c("mpg", "disp", "hp", "wt", "qsec")]
```

A) Sélection Backward automatique

1. Appliquer la méthode de sélection Backward automatique en utilisant la fonction **step()**.
2. Afficher le modèle final **summary(Backward_model)**

B) Sélection Backward manuelle en basant sur **la p-value**.

- Comparer ce modèle avec celui obtenu par la méthode automatique.

C) Appliquez la méthode mixte (stepwise) sur le même jeu de données.

Chapitre 4 : Classification Hiérarchique

La **classification hiérarchique** est une méthode d'analyse de données qui permet de **regrouper des individus (ou objets) similaires** en construisant une hiérarchie de groupes.

Principe :

Le but est de créer une **structure en arbre** appelée : **dendrogramme**
Chaque niveau de l'arbre correspond à un regroupement plus ou moins large.

Le dendrogramme :

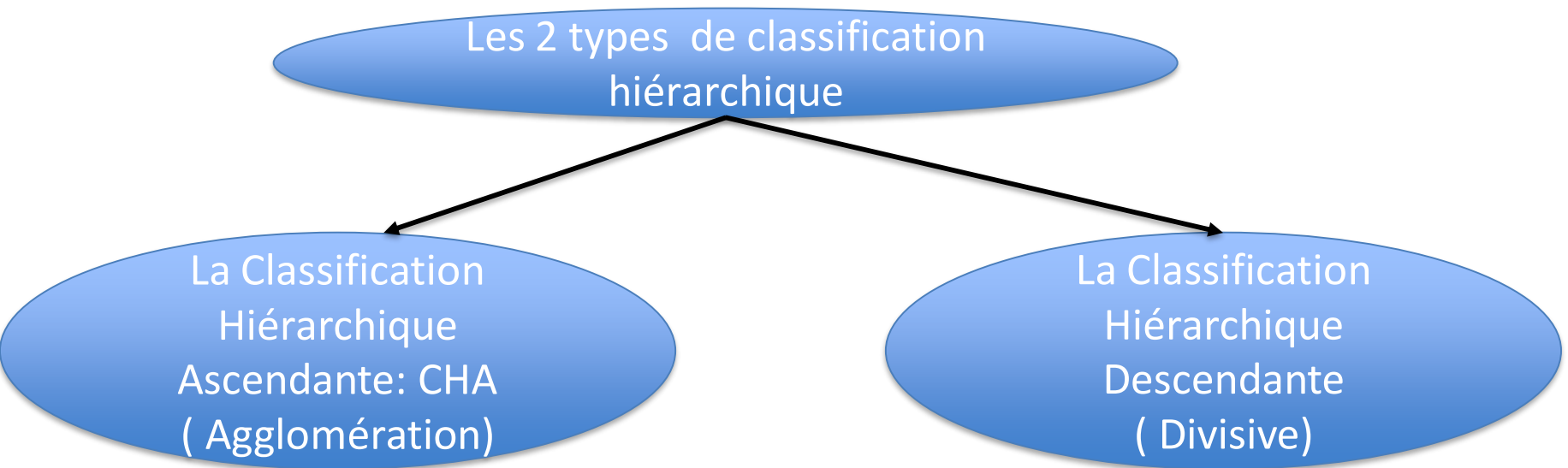
C'est un graphique en forme d'arbre qui montre :

- les étapes de regroupement
- la distance à laquelle les groupes fusionnent

👉 **Plus la fusion est haute → plus les groupes sont différents**

Les types de Classification Hiérarchique :

Il existe deux approches principales pour construire cette hiérarchie, selon que l'on parte des individus isolés ou de l'ensemble du groupe.



La CHA : C'est une méthode dite "**Bottom-Up**" (du bas vers le haut). On commence par considérer chaque individu comme un groupe unique, puis on les fusionne successivement selon leur ressemblance.

Un exemple pratique: Segmentation de clients

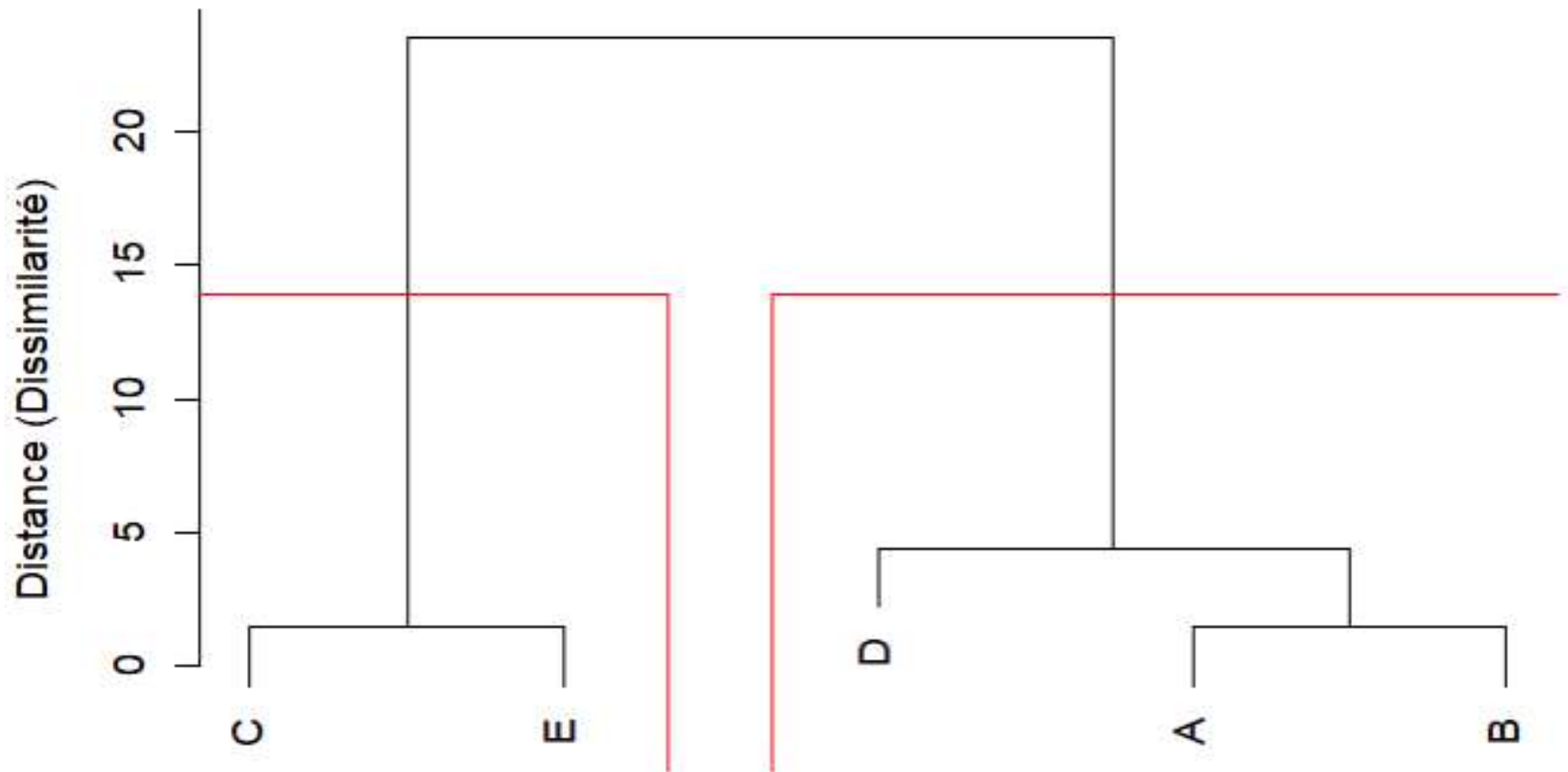
Imaginons que vous analysiez les habitudes d'achat de 5 clients (**A, B, C, D, E**).

- **Étape 0 (individus)**: Vous avez 5 points séparés : (**A**) (**B**) (**C**) (**D**) (**E**)
 - Chaque client est son propre groupe.
- **Étape 1 (Première fusion)**: On remarque que **A** et **B** achètent souvent les mêmes produits.
 - Schéma : $(A + B) \rightarrow \{ A, B \}, (C), (D), (E)$
- **Étape 2 (Deuxième fusion)** : On voit que **C** et **E** sont également très proches l'un de l'autre.
 - Schéma : $\{A, B\} , (C + E) \rightarrow \{ A, B \}, \{C, E\}, (D)$
- **Étape 3 (Troisième fusion)** : On voit que **D** et le couple $\{A, B\}$ sont également très proches l'un de l'autre.
 - Schéma : $(A, B, D) , \{ C + E \} \rightarrow \{ A, B, D \}, \{C, E\}$
- **Étape 4 (Fusion finale)** : On regroupe les deux blocs restants pour avoir une vision globale.
 - Schéma : $(A, B, D) , \{ C + E \} \rightarrow \{ A, B, D, C, E \}$

Exemple du code Sous R:

```
data_clients <- data.frame(  
  Alimentaire = c(10, 11, 20, 12, 21),  
  Electronique = c(2, 3, 15, 6, 16)  
)  
  
data_clients  
rownames(data_clients) <- c("A", "B", "C", "D", "E")  
data_clients  
  
dist_matrix <- dist(data_clients, method = "euclidean")  
dist_matrix  
  
cah_resultat <- hclust(dist_matrix, method = "ward.D2")  
cah_resultat  
  
plot(cah_resultat,  
  main = "Dendrogramme de segmentation des 5 clients",  
  xlab = "Clients",  
  ylab = "Distance (Dissimilarité)",  
  sub = "Étapes de fusion : (A+B), (C+E), (AB+D), puis Finale")  
  
rect.hclust(cah_resultat, k = 2, border = "red")  
|
```

Dendrogramme de segmentation des 5 clients



Clients
Étapes de fusion : (A+B), (C+E), (AB+D), puis Finale

Classification Hiérarchique Descendante (CHD)

La **classification hiérarchique descendante (CHD)** est une méthode statistique qui permet de **regrouper des individus (ou objets)** en classes homogènes.

☞ Elle fonctionne en **partant d'un seul groupe** contenant tous les individus, puis en le **divisant progressivement** en sous-groupes.

Principe général

Contrairement à la classification ascendante (qui regroupe), ici on **divise** :

1. On commence avec **tous les individus dans une seule classe**
2. On cherche la **meilleure division possible**
3. On coupe le groupe en **deux sous-groupes**
4. On recommence sur chaque sous-groupe
5. On s'arrête quand les groupes sont suffisamment homogènes

Critère de division

La division se fait selon un critère :

- Maximiser la **différence entre les groupes**
- Minimiser la **différence à l'intérieur des groupes**

On utilise souvent :

- Distance
- Variance etc.

Un exemple Pratique : Segmentation des 5 clients

On considère 5 clients avec leur **montant d'achat mensuel (en Dirhams)** :

Client	Dépense (Dirhams)
A	6 000
B	5 950
C	5 000
D	3 000
E	2 500

Étape 1 : Groupe initial

Au départ, **tous les clients sont dans un seul groupe** :

{A, B, C, D, E}

Étape 2 : Première division

On cherche à séparer les clients en **deux groupes homogènes**.

On remarque deux catégories naturelles :

- Clients qui dépensent beaucoup :

{A, B, C}

- Clients qui dépensent peu :

{D, E}

Étape 3 : Deuxième division

On peut continuer à diviser :

Groupe {A, B, C}

- A (6 000) et B (5 950) sont proches

- C (5 000) est un peu plus bas

Division :

- {A, B}

- {C}

Groupe {D, E}

- D (3000) et E (2500) sont déjà proches
donc on peut garder ce groupe tel quel

Exercice d'application

Client	Dépense (Dirhams)
A	6 000
B	5 950
C	5 000
D	3 000
E	2 500

1. Créer le **dataframe** des clients à partir du tableau ci-dessus.
2. Installer et charger le **package cluster** afin d'utiliser la fonction **diana ()** .
3. Effectuer la **classification hiérarchique descendante (CHD)**.

```

install.packages("cluster")
library(cluster)

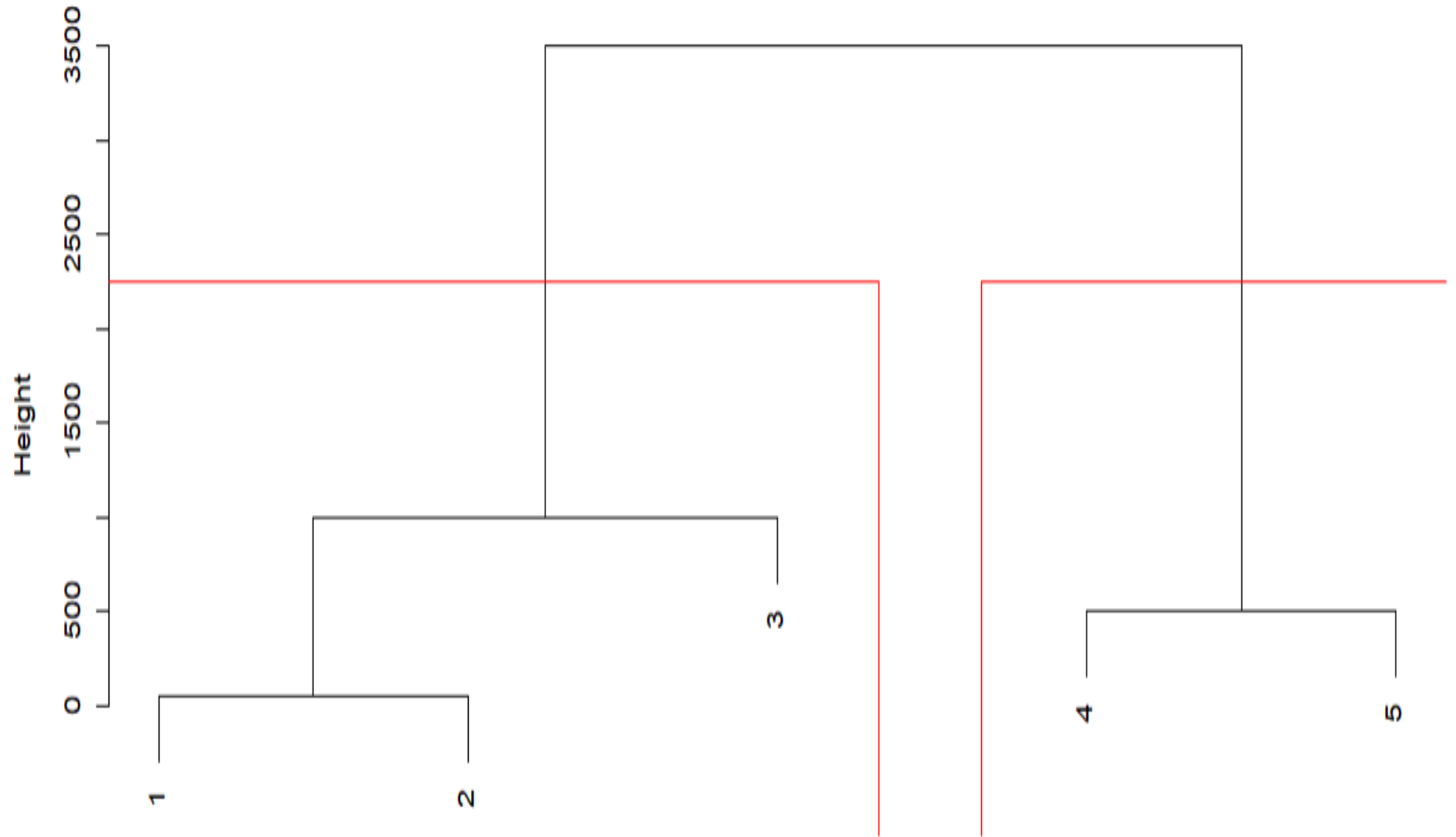
# Création du dataset
clients <- data.frame(
  Client = c("A", "B", "C", "D", "E"),
  Depense = c(6000, 5950, 5000, 3000, 2500)
)
rownames(clients) <- clients$Client
clients <- clients[,-1]

# Standardisation
#clients_scaled <- scale(clients)
# 4. Classification hiérarchique descendante (DIVISIVE)
diana_res <- diana(clients)
# Convertir diana en hclust
chd <- as.hclust(diana_res)
# 5. Dendrogramme
plot(chd, main="Dendrogramme des fruits via la chd")

# 6. Découpage en 2 groupes
rect.hclust(chd, k=2, border="red")

```

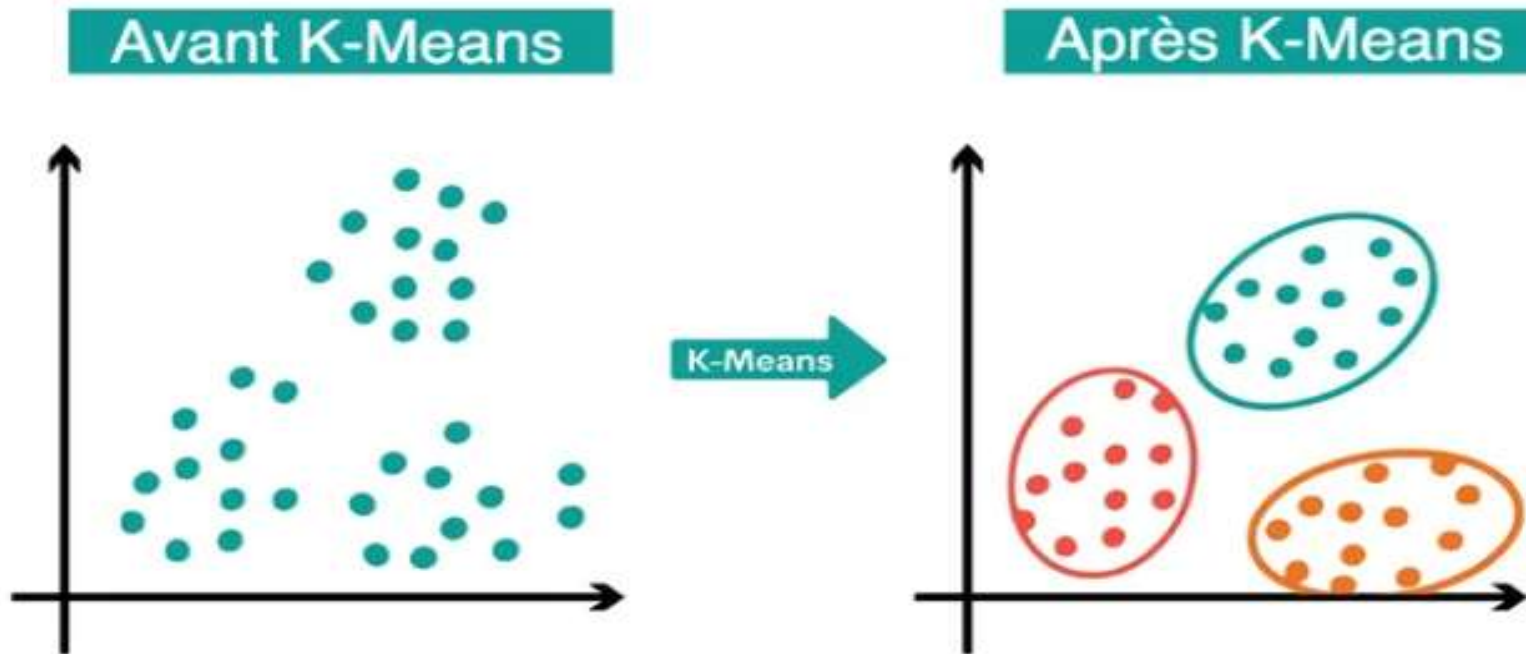
Dendrogramme des fruits via la chd



Chapitre 5 : Classification Non Hiérarchique(k-means)

La classification non hiérarchique, ou partitionnement, regroupe des données en un nombre prédéfini **k** de clusters homogènes sans structure d'arbre.

L' algorithme **K-means**(Les K-moyennes)
c'est l'algorithme le plus populaire. Son but est de grouper les données autour de 'centres' de gravité appelés centroïdes.



Principe de fonctionnement des Kmeans

1. Initialisation :

- Choisir le nombre de clusters (k) souhaité
- Sélectionner K points comme centres initiaux (centroïdes)

2. Attribution des points au cluster le plus proche:

- Calculer la distance entre chaque point et les centroïdes
- Assigner chaque point au cluster avec le centroïde le plus proche

3. Mettre à jour des centroïdes:

- Calculer les nouveaux centroïdes en prenant la moyenne des points de chaque cluster.

4. Répéter les étapes 2 et 3 jusqu'à convergence:

- Réattribuer les points aux clusters les plus proches
- Mettre à jour les centroïdes jusqu'à ce que les changements deviennent minimes.

Critère d'arrêt:

- Convergence: lorsque les points ne changent plus de cluster
- Nombre maximum d'itérations atteint.

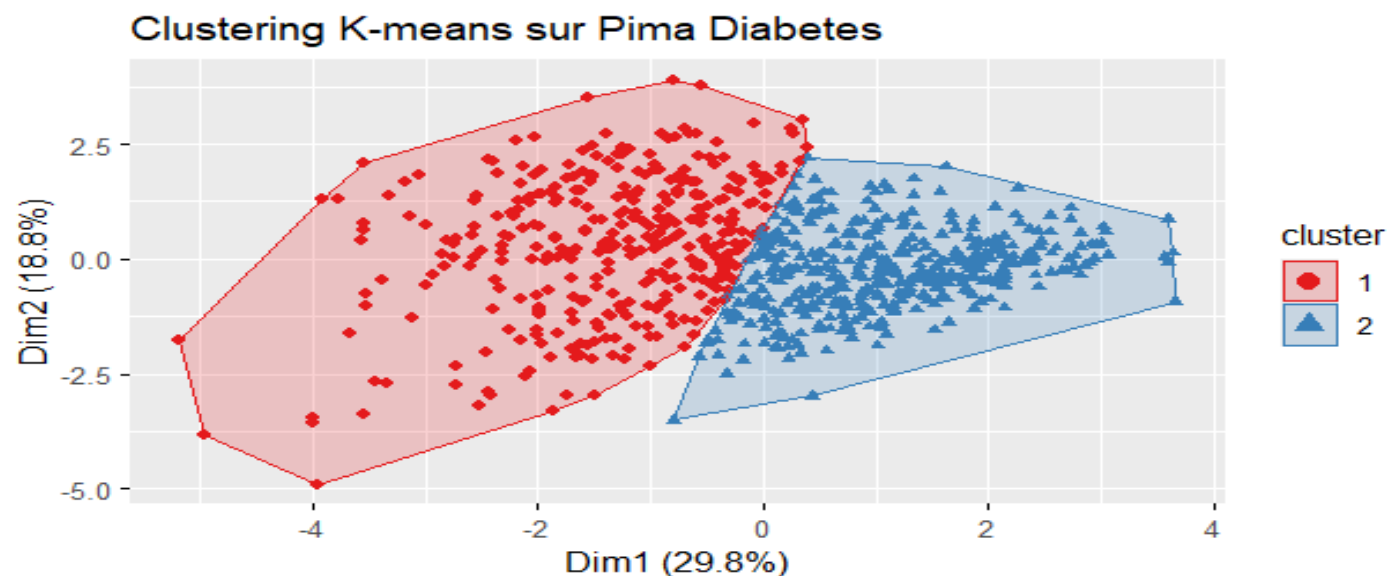
Résultats des k-means:

- Les données sont regroupées en K clusters distincts
- Chaque cluster est caractérisé par son centroïde.

Exercice d'application

1. Importez les **données Pima Indians Diabetes**.
2. **Vérifiez** la **présence** de valeurs manquantes.
3. **Standardisez** les **données**.
4. Choix du nombre de clusters (K)
 - La **méthode** du coude.
 - La **méthode** de la silhouette.
5. **Appliquez** l'algorithme **K-means**.
6. **Affichez** les clusters à l'aide de **fviz_cluster**

Résultat



Le code correspondant

```
library(factoextra)
diabete<-read.csv("C:\\Users\\pc\\Desktop\\AnalyseDonnees\\MyData\\diabet
head(diabete)

# Phase de pretraitement
colSums(is.na(diabete))
df_scaled <- scale(diabete[, -9])
# Méthode du Coude
fviz_nbclust(df_scaled, kmeans, method = "wss") +
  labs(subtitle = "Méthode du Coude")

# Méthode de la Silhouette (mesure la qualité de séparation)
fviz_nbclust(df_scaled, kmeans, method = "silhouette") +
  labs(subtitle = "Méthode de la Silhouette")

set.seed(42)
km_pima <- kmeans(df_scaled, centers = 2, nstart = 25)

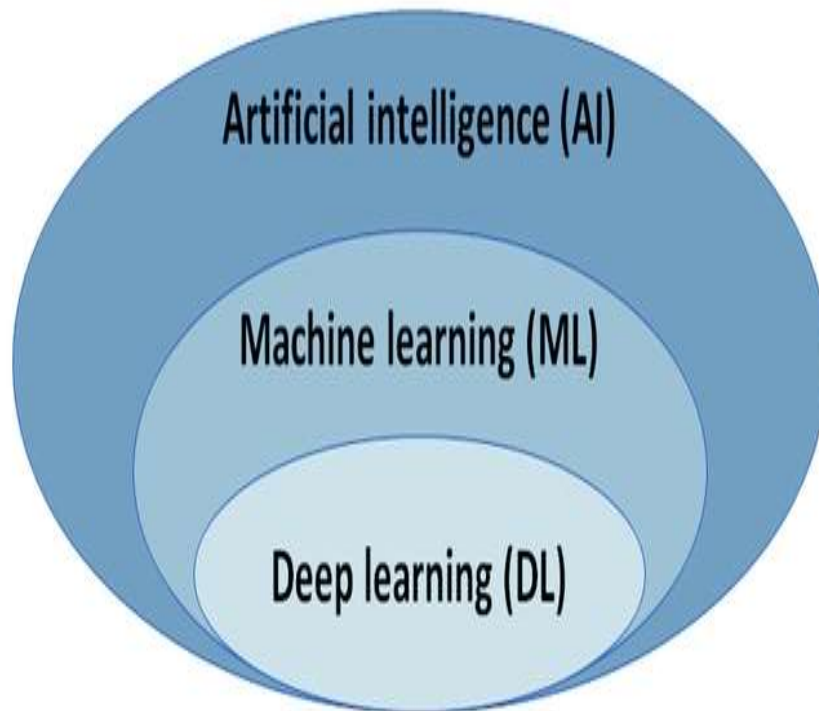
# Visualisation en 2D
fviz_cluster(km_pima, data = df_scaled,
              palette = "Set1", |
              geom = "point",
              main = "Clustering K-means sur Pima Diabetes")
```

Chapitre 6 : Fondements et Enjeux du Machine Learning

Quelle est la relation entre l'analyse de données (AD) et le machine Learning (ML) ?

AD : C'est le socle. Elle fournit les outils exploratoires et statistiques pour comprendre les données avant, pendant, et après le processus de **ML**.

Qu'est ce que le Machine Learning (ML) ?



IA : Le concept global.

Toute technique permettant à une machine d'imiter un comportement humain intelligent.

ML: Un sous-ensemble de l'IA.

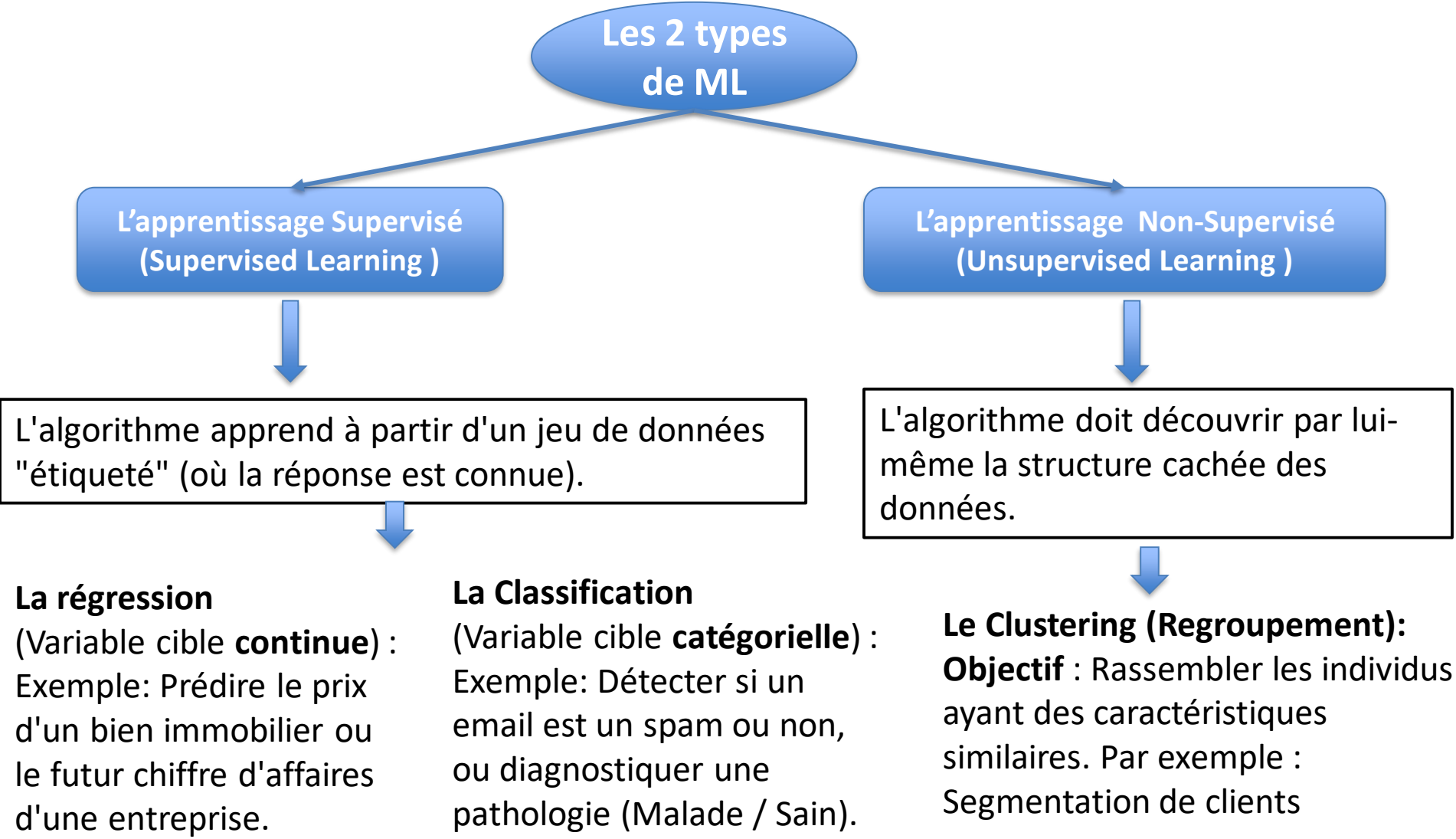
La machine apprend à partir des données sans programmation explicite des règles.

DL: Un sous-ensemble du ML

utilisant des réseaux de neurones profonds (inspirés du cerveau).

Les types des algorithmes de ML

Le **Machine Learning** s'articule autour de trois types, dont deux sont au cœur de notre cours d'Analyse de Données (AD).



Exercice d'application 1:

À partir du dataset **diabetes.csv** , appliquez un arbre de décision en R pour prédire si un patient est diabétique (**Outcome**, **0** = non diabétique, **1** = diabétique (**variable cible**))

- 1) Chargez les packages **rpart** , **caret** et **e1071**
- 2) Importez les données et vérifiez les valeurs manquantes
- 3) Divisez le dataset en **80% train / 20% test**
- 4) Entraînez un arbre de décision avec **minsplitlevel = 20** et **cp = 0.01**
- 5) Évaluez le modèle via la **matrice de confusion** et l'**accuracy**
- 6) Comparez l'accuracy train et test pour détecter un éventuel **surapprentissage**

Exercice d'application 2:

À partir du dataset **diabetes.csv**, appliquez un algorithme de **Support Vector Machines (SVM)** en R pour prédire si un patient est diabétique (**Outcome : 0 = non diabétique, 1 = diabétique (variable cible)**).

- 1) Chargez les packages **rpart**, **caret** et **e1071**
- 2) Importez les données et vérifiez les valeurs manquantes
- 3) Divisez le dataset en **80% train / 20% test**
- 4) Entraînez l'algorithme de SVM
- 5) Évaluez le modèle via la **matrice de confusion** en interprétant les résultats
- 6) Donnez l'accuracy du modèle sur les données de test